LLM4Rec Homework Report — Assignment A04

Student: Anna Grishkina

Team: Individual

Date: 2026-10-08

Assignment: A04 — Association Rules (Week 4)

Live app: https://af851574-oss.github.io/RecSys-LLMs/week4/

Abstract

I implemented Apriori and rule generation in the instructor's Week 4 starter, which contains the UCI Online Retail dataset cleaned to 17,080 invoice baskets and 3,653 items. The page's own test harness went from 2 passed / 9 pending to 11 passed, both in Node and in a real headless browser. I chose a minimum support of 1% and a minimum confidence of 50%; this gives 1,219 frequent itemsets and 456 rules, all with lift above 1. The rule worth investigating is the Regency teacup trio (390 baskets, confidence 72.09%, lift 22.35). The rule to reject is *lunch bag apple design → white hanging heart t-light holder* (confidence 13.76%, lift 1.20), which mostly repeats the popularity of the store's most frequent item. All key numbers were recomputed independently from the raw baskets.

Index Terms — association rules, Apriori, support, confidence, lift, market basket analysis

1. Introduction

Problem statement. Treat each invoice as one basket, find itemsets that appear together often, turn them into rules A → B, and decide which rules are worth acting on. The starter [1] provided the data, page, and test harness, and left seven functions as `TODO(hw4)` stubs.

Motivation. Rules like "customers who bought A also bought B" drive cross-sell slots and bundles [2]. The risk is that a rule looks strong only because one item is popular, or because the evidence comes from a small group of baskets.

Concrete example. The white hanging heart t-light holder (`85123A`) appears in 1,959 of 17,080 baskets (11.47%). Any rule ending in it starts with about 11% confidence before any real association is involved.

Contributions.

- Implemented basket deduplication, itemset counting, support / confidence / lift with guarded denominators, level-wise Apriori, and two-direction rule generation.
- Ran a 4 × 3 threshold sweep and justified the final thresholds with its numbers.
- Selected one useful and one misleading rule with full counts and checked both independently.

2. Related Work

Association rules and the support–confidence framework come from Agrawal, Imieliński, and Swami [3]. Apriori uses the downward-closure property: every subset of a frequent itemset is frequent, so a candidate with an infrequent subset can be skipped. FP-Growth mines the same itemsets without generating candidates [4]. I used Apriori because the starter already provides an inverted index for counting, and the dataset is small enough that Apriori finishes in about 1.3 s at my thresholds. The data is the UCI Online Retail dataset [5], introduced by Chen, Sain, and Guo [6]. The starter readme gives that paper's venue as *Journal of Cases on Information Technology*; Crossref lists it in the *Journal of Database Marketing & Customer Strategy Management*, so I cite the Crossref record.

3. Method

Basket matrix. Each invoice is a set of stock codes. Duplicate lines in one invoice are counted once, and descriptions are only labels.

Metrics. For N baskets: support(A → B) = count(A ∪ B)/N, confidence(A → B) = count(A ∪ B)/count(A), lift(A → B) = confidence/(count(B)/N). A zero denominator returns `defined: false` instead of `Infinity` or `NaN`.

Mining. Level-wise Apriori: frequent single items first, then each level joins frequent (k−1)-itemsets that share a prefix, prunes candidates that have an infrequent (k−1)-subset, and counts the rest through the starter's inverted index.

Rules. Every frequent itemset with at least two items is split into every non-empty antecedent / consequent pair, so A → B and B → A are both generated. A rule is kept when its confidence passes the threshold; the analysis keeps only rules with lift > 1.

Tools and libraries.

- Plain browser JavaScript (the page runs from `file://` with no build or server)
- Node.js v24.10.0 for `week4/test.mjs` and `week4/analysis.mjs` (`node:vm` over the real files)
- Google Chrome 152.0.7977.83, driven through the Chrome DevTools Protocol
- Python 3 for my independent recomputation
- GitHub Pages for hosting

Thresholds. Minimum support 1% (171 baskets) and minimum confidence 50%; justified by the sweep in Section 4.

4. Experiments

Setup. With the stubs untouched, the harness reported 2 passed / 0 failed / 9 pending, as the readme states. After the implementation I ran the miner at four support levels and three confidence levels.

Min support	Frequent itemsets (by size)	Rules @30%	Rules @50%	Rules @60%	Mining time
0.5%	5,255 (1,320 / 2,206 / 1,161 / 474 / 88 / 6)	10,723	5,812	3,958	3.2 s
1%	**1,219 (691 / 413 / 109 / 6)**	950	**456**	238	**1.3 s**
2%	294 (242 / 51 / 1)	96	36	20	0.3 s
3%	115 (109 / 6)	12	6	5	0.1 s

Every rule in every cell has lift > 1. At 1% / 50% the table is still readable, it keeps 109 three-item and 6 four-item itemsets for bundles, and mining takes about 1.3 s. At 2% only one three-item set survives, so almost every bundle disappears; at 0.5% the list grows to 5,812 rules, many of them from small groups of baskets.

Hand-computed rule. HERB MARKER THYME (22916) → HERB MARKER ROSEMARY (22917): count(A) = 186, count(B) = 190, count(A ∪ B) = 176, N = 17,080. Support = 176/17,080 = 1.03%; confidence = 176/186 = 94.62%; lift = 0.9462/(190/17,080) = 0.9462/0.011124 = 85.0617. The page, the Node pipeline, and my raw-basket scan give the same values.

Selected rules.

Rule	count(A)	count(B)	count(A ∪ B)	Support	Confidence	Lift
Useful: GREEN + ROSES REGENCY TEACUP → PINK REGENCY TEACUP	541	551	390	2.28%	72.09%	22.35
Reject: LUNCH BAG APPLE DESIGN → WHITE HANGING HEART T-LIGHT HOLDER	865	1,959	119	0.70%	13.76%	1.20

Direction. Thyme → rosemary has confidence 94.62% and rosemary → thyme 92.63%; the teacup rule gives 72.09% and 70.78% in reverse. Lift is 85.0617 and 22.3462 in both directions. In the browser, the *Reverse direction* button shows the same change.

Verification performed.

1. `node week4/test.mjs` — all 13 checks pass: the starter harness 11/0/0; N, item count, and count(85123A); both lecture examples (support 37.5%, confidence 60%, lift 0.96; 60% vs 50% with equal lift); zero denominators; every itemset recounted; downward closure; every rule passes both thresholds; lift symmetry; the hand-computed rule.
2. Headless Chrome through the DevTools Protocol actually clicked the buttons: *Run tests* showed “pass 11, fail 0, pending 0”; at 1% / 50%, *Run rules* listed 456 rules in about 1.1 s; the detail and reverse views showed the values above. At the loosest setting (0.5% / 10%) the page listed 19,058 rules in 2.9 s.
3. `node week4/analysis.mjs` printed the same counts in two runs, and `node --check` passed.
4. I wrote a separate Apriori in Python over the raw baskets. It reproduced 294 itemsets / 36 rules at 2% and 1,219 itemsets / 456 rules at 1%, with zero rules at lift ≤ 1, and all counts in the table above.
5. Only the seven stub functions differ from the starter; `transactions.js`, `index.html`, `style.css`, and `readme.md` are byte-identical.

5. Discussion

Useful rule. Baskets with the green and roses Regency teacups contain the pink one in 72.09% of cases, while only 3.2% of all baskets do (551/17,080). The joint count is 390, against about 17.5 expected under independence, so the lift of 22.35 rests on hundreds of baskets. The action I would test is a "complete the set" slot for the pink teacup when the other two are in the cart, or a three-cup bundle. Confidence alone would not justify this; the lift shows that the pattern is not explained by the pink cup's own popularity.

Rejected rule. A naive reading says "lunch-bag buyers also buy the heart t-light holder", so merchandise them together. But the holder is already in 11.47% of all baskets, and the rule's 13.76% is barely above that. Under independence about 99 joint baskets are expected; we see 119. Even that small excess is uncertain: the 95% interval for the confidence is 11.6%–16.2%, which puts the lift between 1.01 and 1.41. Placing the two products together would mostly reach shoppers who buy the popular holder anyway.

Why no "high confidence, lift near 1" rule exists here. With the most frequent item at 11.47%, any rule with confidence ≥ 50% has lift ≥ 0.5/0.1147 = 4.36. The popularity trap in this dataset therefore appears at low confidence. A second warning sign is the herb-marker family: rules with 99% confidence and lift around 90 come from 132 baskets that contain all six herb markers, which looks like one set bought as a whole, not a cross-sell discovery.

Threshold trade-off. Raising support removes itemsets before they are counted: once a pair drops below the threshold, every larger set containing it is pruned, which is why 3% leaves only six pairs and no three-item set. Confidence never re-mines; at 1% support, moving from 30% to 60% keeps the same 1,219 itemsets and cuts the rules from 950 to 238. Lower thresholds give more rules, but more of them rest on small groups of baskets.

Association is not causation. The teacups are one product line, and the dataset contains many wholesale customers who restock a full range. The customer segment and the catalog layout can produce the co-occurrence, so promoting the green and roses cups may not sell more pink cups. High lift only says that the items appear together more often than chance in past invoices.

Validation before action. The exported data has no usable per-basket timestamp, so I could not check whether the rule holds in a later period, and I did not simulate one. With dated invoices, I would mine on an earlier window and accept the rule only if support stays near 1%, confidence stays inside its training interval, and lift stays far above 1 in the later window. The smallest real test is an A/B experiment: show the "complete the set" slot to a treatment group, compare the pink-cup attach rate with a control group, and ship only if the difference is significant. Both require data that the starter does not contain.

What worked.

- Reusing the starter's inverted index for every count kept the table, the detail panel, and the tests consistent.
- Driving the real browser exposed the full click path, not only the functions.

What surprised me or did not work.

- The 50% confidence threshold already guarantees lift ≥ 4.36 here, so the classic "high confidence, lift ≈ 1" example is impossible in this data.
- The starter readme's citation for the dataset paper does not match Crossref.

Next improvement. Use the original `InvoiceDate` to test rule stability over time, and compare against FP-Growth at 0.5% support, where Apriori takes about 3 s.

6. AI Usage Disclosure

AI tools used. Pi Coding Agent 0.85.1; the implementation session ran on the GLM-5.3 model through the Z.ai provider.

How I used AI. I wrote the task with my decisions (Apriori, threshold sweep, which candidate rules to print, the browser check) and gave it to a coding agent in a separate session. I also used AI to draft the report text.

What I verified myself. I recomputed the thyme → rosemary rule by hand, reran the tests, the analysis, and the browser check, reimplemented Apriori in Python and matched the itemset and rule counts at 1% and 2%, computed the expected count and confidence interval of the rejected rule, and checked every reference through Crossref or by opening it.

What I trusted without verification. I did not check the instructor's data cleaning against the original spreadsheet.

Session log. The attached `argrishkina_a04_session.json` is a privacy-redacted audit generated from the task-only implementation session. It contains tool calls, file changes, and milestones, but excludes conversational content and replaces local account identifiers. It is not an unmodified Pi export. The prompts and notes are in `week4/prompt.md`.

References

[1] J. S. Jin, "HW4 — Association Rules (week4)," *GitHub*, 2026. [Online]. Available: https://github.com/dryjins/RecSys-LLMs/tree/main/week4. [Accessed: 2026-10-08].

[2] S. Jin, "Association Rules," lecture slides, LLM4Rec course, HSE University, 2026.

[3] R. Agrawal, T. Imieliński, and A. Swami, “Mining association rules between sets of items in large databases,” in *Proc. 1993 ACM SIGMOD Int. Conf. Management of Data*, 1993, pp. 207–216, doi: 10.1145/170035.170072.

[4] J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation,” in *Proc. 2000 ACM SIGMOD Int. Conf. Management of Data*, 2000, pp. 1–12, doi: 10.1145/342009.335372.

[5] D. Chen, “Online Retail,” UCI Machine Learning Repository, 2015, doi: 10.24432/C5BW33. [Online]. Available: https://archive.ics.uci.edu/dataset/352/online+retail. [Accessed: 2026-10-08].

[6] D. Chen, S. L. Sain, and K. Guo, “Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining,” *J. Database Marketing & Customer Strategy Management*, vol. 19, no. 3, pp. 197–208, 2012, doi: 10.1057/dbm.2012.17.